NewUM Data Team

System Design Document

Owners

Michał Mucha — Team Lead, Lead Data Scientist

Oleksandr Nykonenko — Senior Data Engineer

Arben Osmani — Senior Data Engineer

Carlos Carrillo — Senior Data Engineer

7000 Central Parkway, Suite 1750, Atlanta, GA 30328

info@oncohealth.us | oncohealth.us

Objective 2
High Level Design 3
Detailed Design 5
Cases domain 14
Metrics, Monitoring & Alerting 19
Deployment, Integration and Rollout 20
Scalability 21
Testing 22
Data Migration 22
Data Management 22
Security and Compliance Concerns 23
Scope / Constraints 24
Glossary 25

Objective

The NewUM Data Team provides the data architecture underpinning the NewUM prior-authorization (PA) system for oncology and specialty-drug cases. The platform ingests operational data from multiple upstream sources, applies medallion-layer (Bronze/Silver/Gold) transformations in Databricks, and delivers analytically and operationally ready datasets to downstream consumers.

Success is measured when all ingestion pipelines run within defined SLAs, data quality gates enforce zero silent propagation of bad data, downstream consumers receive certified gold datasets on schedule, and the platform supports 10x current PA case volume without architectural change.

Requirements

- Ingest all registered data sources into Databricks Bronze Delta tables per documented SLAs.
- Implement medallion architecture (Bronze/Silver/Gold) with enforced DQ gates at each layer.
- Support near-real-time CDC ingestion from NewUM operational DB.
- Deliver Gold-layer datasets for analytic downstream.
- Enforce HIPAA-compliant PII tokenization at the Bronze write layer.
- Maintain data models and dictionaries for the Drugs, Eligibility and Provider domains.
- Provide bidirectional data exchange between NewUM and Databricks with latency appropriate to each use case.
- Prohibit all analytic queries against operational databases; all analytics served from Databricks Gold (or Silver where approved).

Success Criteria

- Eligibility and Provider data is available for NewUM.
- All ingestion pipelines meet defined Data-in-Bronze SLAs (CDC: near-real-time; batch feeds: per schedule).
- Zero DQ failures propagate silently to Gold; quarantine rate < 1% per batch in steady state.
- Gold datasets available to analytic, BI, payer, and finance consumers within agreed freshness windows.
- All schema changes follow the versioning and data contract protocol; no uncoordinated breaking changes.

High Level Design

The NewUM Data Platform follows a medallion lakehouse architecture on Azure Databricks, with ADLS Gen2 as the storage layer. Data flows from NewUM operational systems through Bronze (raw ingestion), Silver (cleansed/conformed), and Gold (business-facing) tiers. Apache Airflow on AKS orchestrates all batch and hybrid pipelines. Unity Catalog governs access control, lineage, and data discovery across all layers.

Layer	Purpose	Data Quality Bar
Bronze	Raw ingestion; full-fidelity, append-only	Schema valid; no transforms; PII tokenized at write
Silver	Cleansed, deduplicated, common data model across customers, conformed to NewUM domain model	Null rates < thresholds; RI enforced; quarantine on failure
Gold	Business-facing aggregates, reporting models, feature sets	All DQ gates passed; certified datasets only

Ingestion Source Registry

Every data source must be registered below before its pipeline is promoted to production. The registry defines ownership, ingestion pattern, format, and SLA for each upstream feed.

Source	Latency	Key Entities	Notes
NewUM Operational DB	Daily	Cases, Providers, Eligibility, Case Queuing, Configuration	Core operational store; feeds reporting and rules audit
DrugsMS (Matis API)	Daily	MDL, Regimen Library, HCPCS, RxCUI, NDC, GPI, EADL	Source of truth for drug identifiers and regimen definitions
Payer Eligibility Feed	Daily	MemberEligibility, LOB, ProductLine, CoverageSpan, BenefitPackage	Eligibility verification, scope determination, carve-out rules
Provider Master Data	Daily	ProviderNPI, Specialty, Address, TaxID, GroupAffiliation	Ordering/servicing provider validation on case entry

Payer Drug Library (PDL)	Daily	BenefitType, PARequired, OOS, CrossoverFlag, BillingMethod	Drives table placement, PA routing, drug status display
Admin Portal: Regimen Library	Daily	RegimenName, DiagnosisGroup, DrugDose, AutoApprovedFlag	Published chemo regimens; dosing pre-population
Admin Portal: ICD10 / Dx Library	Daily	ICD10Code, DiagnosisGroup, TreatmentTypeApplicability	ICD10 search, grouper derivation, treatment-type filtering
Admin Portal: NP Drug Rules	Daily	NP Drug, Preferred Alternatives, Rule Type, LOB, Dx Group	NP drug flagging, mandatory/voluntary enforcement
PBM / Formulary Data	Daily	FormularyStatus, QuantityLimit, PAStatus, DrugNDC	Pharmacy formulary logic, quantity limits, PA triggers

Outgoing Data Streams

All analytic traffic is served exclusively from the Databricks Gold layer (or Silver where explicitly approved). Direct queries against NewUM PostgreSQL or any operational store for analytical purposes are strictly prohibited. Downstream consumers and their delivery patterns:

Consumer	Delivery	Format	Frequency
Analytics/BI	Databricks SQL	Delta/SQL View	On-demand + daily
Payer Reporting	Scheduled file → ADLS	CSV/HL7/JSON	Daily/Monthly
Finance/Billing	Scheduled Delta export	Parquet/CSV	Daily
Clinical Ops Dashboard	Databricks SQL	Delta/SQL View	Daily
Compliance/Audit	ADLS Gen2 immutable	Parquet + JSON manifest	Monthly
ML / Data Science	Feature Store	Delta feature table	Daily

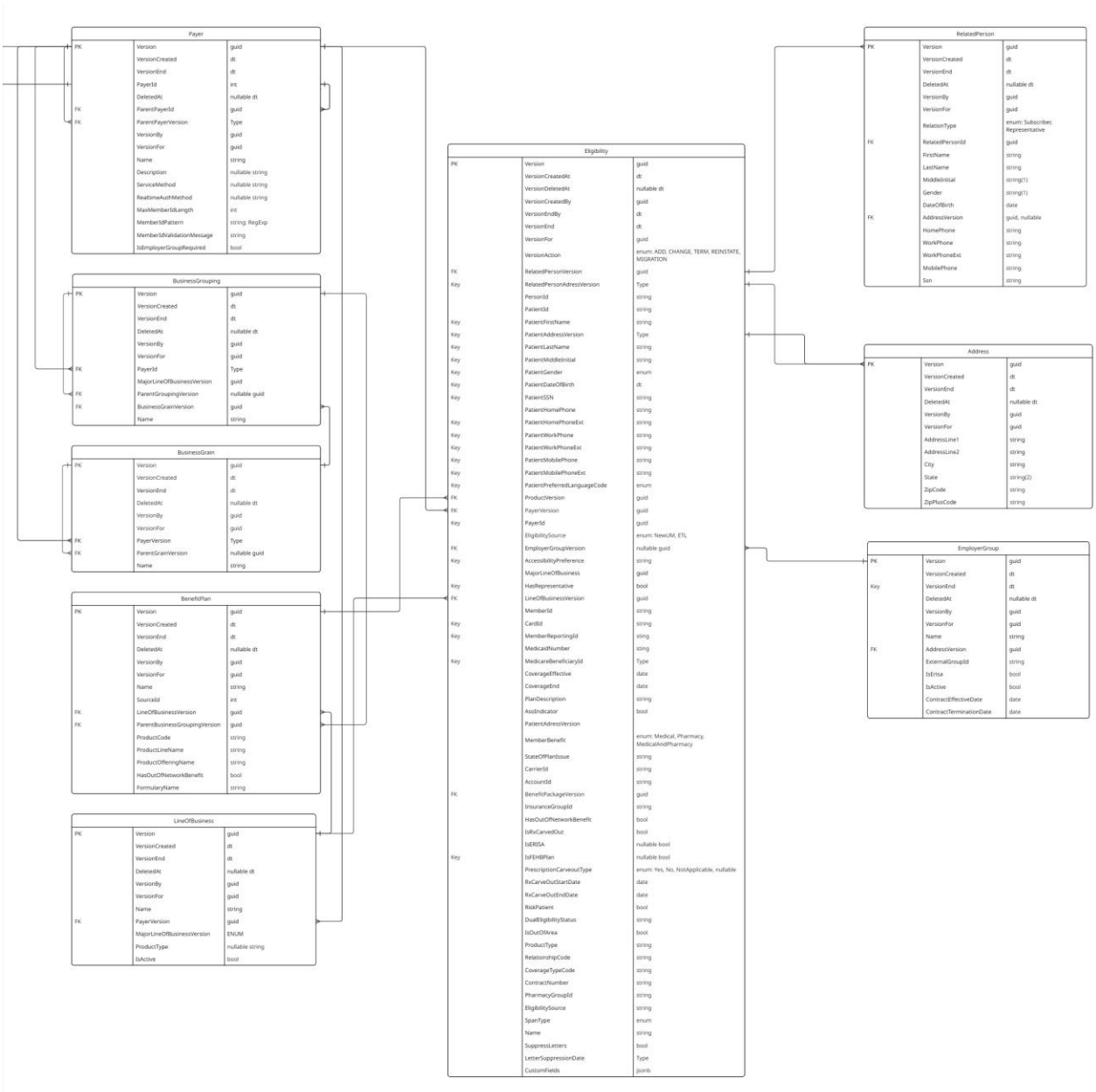
Detailed Design

Eligibility Domain

Data model ER diagram: Link to documentation

Data dictionary: Link to documentation

The Payer Hierarchy Chain: Link to documentation



Payer Hierarchy

The schema encodes two converging hierarchies rooted at Payer. The product classification hierarchy — MajorLineOfBusiness → LineOfBusiness → BenefitPlan — drives PDL lookups, NP rule evaluation, and scope determination. Payer admin hierarchy — Payer → MajorLOB → BusinessGrouping → BenefitPlan — carries admin-configured rules for scope, formulary, and NP evaluation. Product is the convergence point for both, making it the structural anchor for all downstream clinical and administrative rule resolution.

Member & Related Person

Person records are keyed by MemberID + Payer and serve as the PHI container — name, DOB, gender, address, and related identifiers. RelatedPerson models the policyholder as a distinct entity, while EmployerGroup captures group sponsorship context. ListOfMembers bridges the member side to the eligibility core, likely supporting multi-member coverage units or dependent enumeration.

Eligibility Record

Eligibility records are versioned coverage spans linked to a Person and a Product/LOB classification. The entity carries the full set of scope-determining attributes: AccountID, InsuranceGroupID, BenefitPackage, CarrierID, ASO indicator, state of issue, member benefit flags, and PrimaryOrCarveoutType. Scope validation evaluates all of these against the admin hierarchy; failure on any check produces an Out-of-Scope determination, blocking clinical review. No case progresses through member identification, coverage verification, or scope determination until all three steps pass.

Table Purposes

Table	Purpose
Payer	Master record for an insurance payer (e.g., Medicare, Medicaid, commercial). Holds identity, service configuration, and member ID validation rules.
BusinessGrouping	Logical grouping of plans/products under a payer, organized by major line of business. Supports hierarchical groupings (self-referencing via ParentGroupingVersion).
BusinessGrain	Lowest-level classification unit within a payer's business hierarchy. Acts as a leaf node below BusinessGrouping.
BenefitPlan	A specific insurance product/plan tied to a line of business and business group. Carries formulary, offering, and network benefit flags.
LineOfBusiness	A mid-tier classification sitting between MLOB and Product. It represents a meaningful business subdivision within a major category.
Eligibility	Core transaction table. Represents a member's coverage span — effective dates, benefits, carve-outs, plan type, and eligibility source. One record per coverage action.

RelatedPerson	Demographic record for a person related to a member (subscriber or authorized representative). Linked to a Person record via PersonId.
Address	Normalized address entity. Referenced by Person and RelatedPerson via FK.
EmployerGroup	Employer group sponsoring coverage. Linked to Payer and holds ERISA flag, contract dates, and external group ID for payer cross-reference.

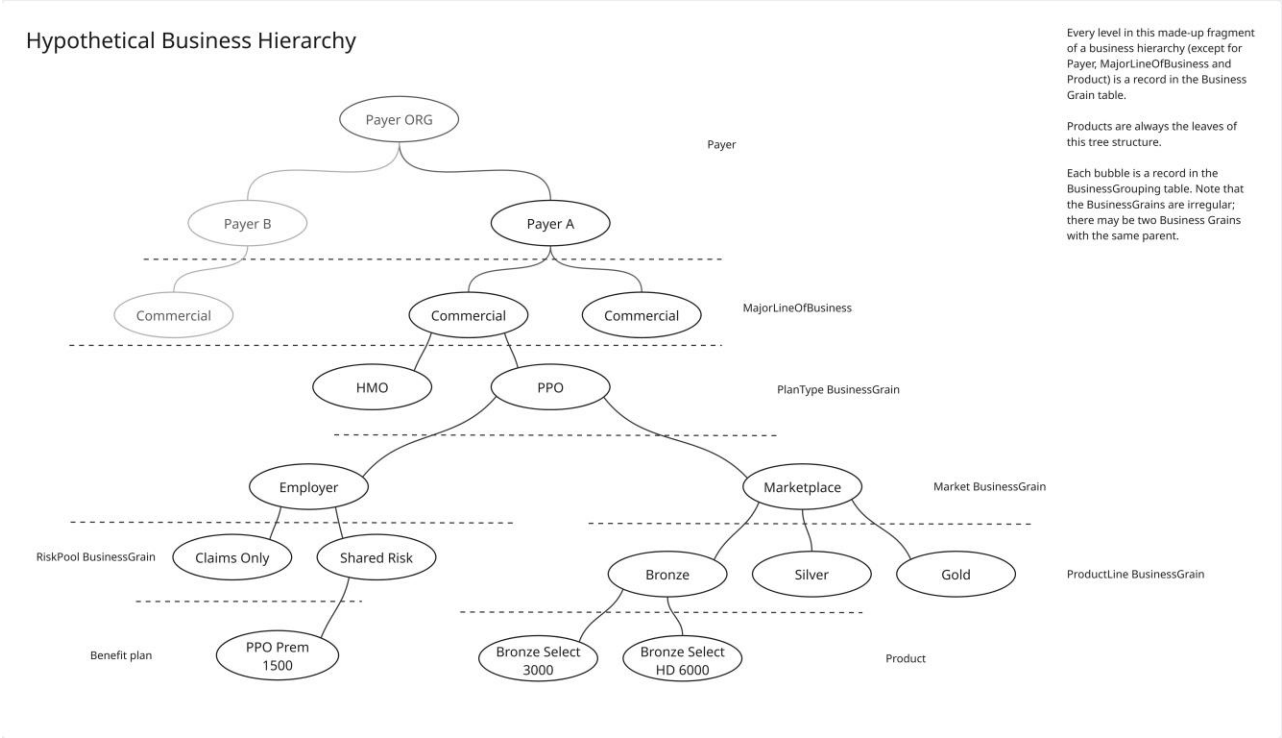
Versioning Strategy

All versioned tables use VersionCreatedAt (when this version becomes current) and VersionEnd (when superseded, nullable - NULL means current). When a new version arrives for the same entity, the ETL sets VersionEnd on the previous current row and inserts the new row. This is the one place where an existing row is modified. DeletedAt serves a different purpose - it means the record was deleted and should never surface in UI regardless of time queries. Both VersionEnd and DeletedAt can be set on the same record.

Employer Group vs. Plan Taxonomy: Orthogonal Constructs

EmployerGroup, BusinessGrouping, and BusinessGrain are complementary but orthogonal constructs. BusinessGrouping and BusinessGrain represent the payer's internal product taxonomy — a stable classification hierarchy defining what kind of plan a product is. EmployerGroup represents a specific employer's contractual relationship with the payer — who purchased coverage, under what terms, and for how long. Two employers can purchase the same product (same BusinessGrain) under entirely different contract terms, ERISA status, or external group identifiers; the product taxonomy has no mechanism to capture this variation. Eligibility is the convergence point, holding foreign keys to both, binding a member to a plan of classification and an employer sponsor simultaneously.

Table	BusinessGrouping / BusinessGrain	EmployerGroup
What it models	Payer's internal product taxonomy	A specific employer's contract with the payer
Defined by	The payer (plan design)	The employer (contract negotiation)
Cardinality	Fixed hierarchy — few records, stable	One per employer client — potentially thousands
Changes when	Payer restructures their LOB hierarchy	Employer renews, terminates, or changes groups
Purpose	Classify *what kind of plan* this is	Identify *who is sponsoring* this coverage



Risks

The eligibility model is ~85% validated, but the remaining gaps — business hierarchy mapping, manual overrides, and person identity — are structural blockers that affect rules of evaluation, scope of determination, and multi-payer onboarding. Mid-case eligibility changes and inconsistent payer file behavior add compliance and financial exposure. Dedicated workshops are planned to resolve each item before implementation proceeds.

Manual eligibility overrides have no agreed storage pattern

Clinical staff frequently need to manually create or correct eligibility records (e.g., payer confirms coverage by phone, but the file update takes two weeks). The Onco team and the NewUM team have opposing positions on where this data should live. If manual records go into the ETL-managed table, nightly batch loads will overwrite them. If they go elsewhere, the application needs to merge two data sources at query time. Neither pattern has been designed.

Business impact: Without resolution, intake staff cannot process cases where eligibility data is known-incorrect — a daily occurrence. Cases will stall or be processed against bad data, leading to incorrect denials or approvals.

Working hypothesis: ETL-sourced and manually entered eligibility records live in the **same table**, distinguished by a source flag (e.g., EligibilitySource: ETL | MANUAL). The ETL load logic is built to respect this flag — batch loads only insert/version records where EligibilitySource = ETL and never touch manually-created records. Manual records follow the same versioning pattern (immutable

inserts, VersionCreatedAt, VersionEndAt) and are first-class citizens in the eligibility table. The application queries a single table with no read-time merging required.

What this unblocks: Single query path for the application — no merge logic. The ETL team builds a source-aware load (filter on flag). The intake team can create manual records immediately using the same schema.

What could invalidate it: If the ETL sync mechanism (e.g., Databricks → Postgres via Lakehouse Sync or similar) operates as a full table replacement rather than row-level upsert, it would wipe manual records on every sync cycle. The ETL tooling must support row-level awareness, or the sync must exclude MANUAL-flagged rows. This needs to be validated against the chosen data exposure pattern.

~85% of the eligibility model is validated, but the remaining ~15% carries disproportionate risk

The workshop confirmed that the core eligibility model (versioning, coverage tracking, person demographics, address, employer group) is sound. However, the unresolved 15% — business hierarchy mapping, manual overrides, person identity grain — touches every downstream system: rules engine, scope determination, letters, reporting, and API exposure.

Business impact: There is a risk of perceiving the eligibility model as "nearly done" when the outstanding items are the ones that determine whether the system can operate in a multi-payer production environment. These are not polish items — they are structural.

Working hypothesis: The remaining structural questions will be addressed through a **series of dedicated workshops** between NewUM Data and Onco Data teams, each focused on one specific problem:

1. Business hierarchy mapping with hybrid JSON/SQL prototype.
2. Manual override and ETL coexistence validation.
3. Person for identity grain definition.

Each workshop produces a concrete deliverable - either a validated prototype, a confirmed schema change, or a documented decision. Workshop cadence is weekly until all three are resolved. Progress is tracked against a shared checklist with explicit "done" criteria, preventing these items from drifting as background concerns.

What this unblocks: Gives the unresolved items a structured resolution path with accountability. Prevents the team from moving to implementation on the assumption that these will "work themselves out." Gives business stakeholders visibility into when the eligibility model will be fully confirmed.

What could invalidate it: If workshop participants are unavailable at the required cadence, or if the workshops surface additional unresolved dependencies (e.g., the rules engine team needs to

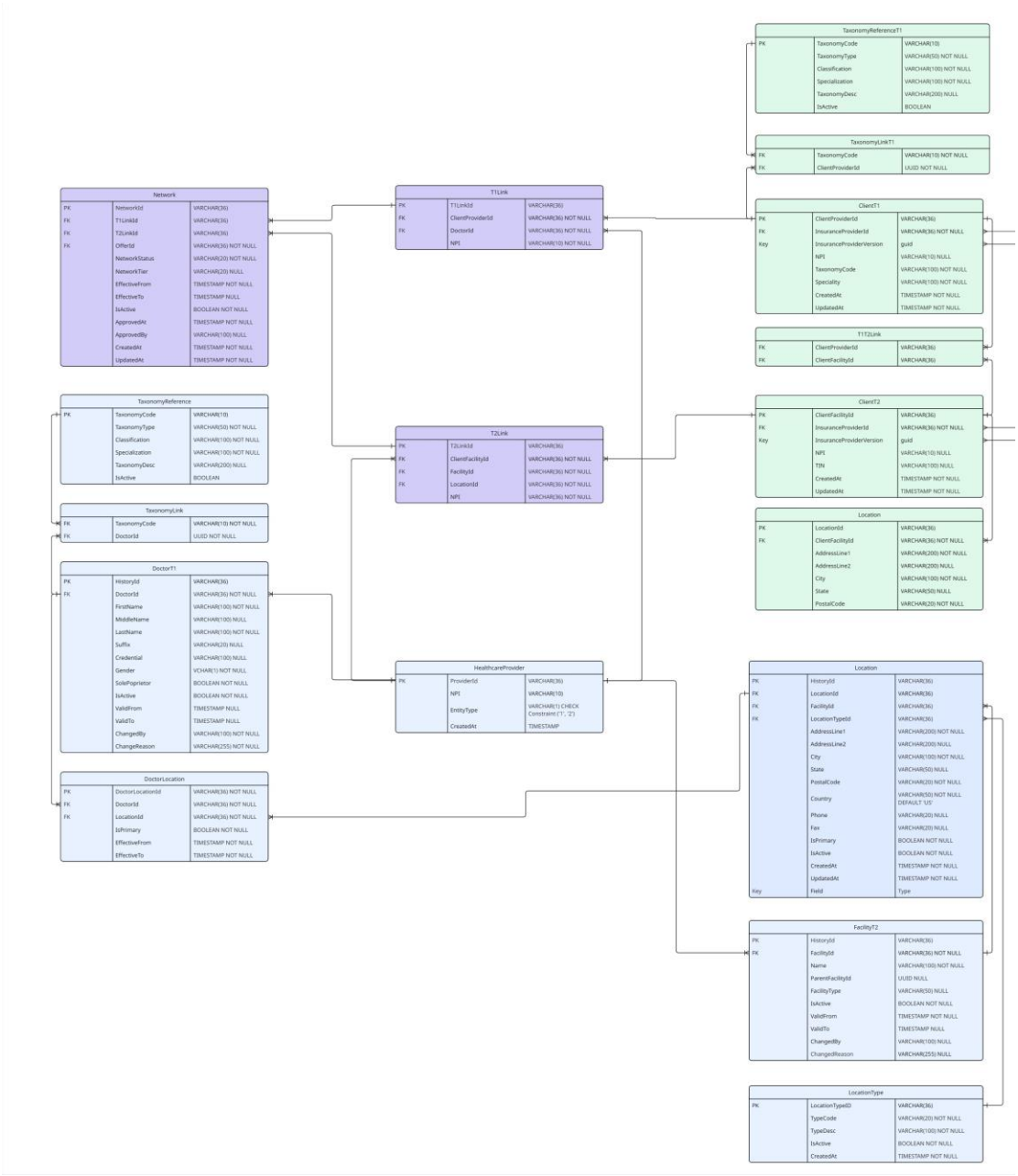
participate but isn't yet engaged). The workshops must include the right people, or they'll produce designs that get rejected downstream.

Provider Domain

Data model ER diagram: Link to documentation

Data dictionary: Link to documentation

NewUM Platform, Provider Abstraction: Link to documentation



Provider Core Identity - NPPES/Government Data

This is the anchor layer. The HealthcareProvider table is the root — every provider must exist here before appearing anywhere else. It holds immutable attributes: ProviderId, NPI and entity type. From here, the model branches into Doctor (Type 1) holding individual attributes like first name, last name, gender, sole proprietor flag, and Facility (Type 2) holding organization name, facility type, and related attributes. The key open question is whether these remain separate tables or merge into one with an entity_type discriminator.

Provider Mutable State (Extensions of Blue Tables)

Attributes that change over time — name changes, credential updates, status changes. The meeting determined these should not live in separate history tables but instead use single-table SCD Type 2 versioning (version_start, version_end, deleted_at), consistent with the eligibility domain. This supports retrospective case lookups where you need to know what the provider looked like at case creation time.

Taxonomy Classification - NPPES/NUCC Reference

Taxonomy codes classify a provider's specialty (e.g., cardiology, internal medicine). It's a many-to-many relationship — one doctor can have multiple taxonomy codes. This requires a bridge table between the provider and a taxonomy reference table. Both Type 1 and Type 2 providers can have taxonomies. However, the group questioned whether taxonomy is even needed in the NewUM application, since historically it's only been used for pre-UM filtering. Pending BA confirmation — default position is to exclude from V1.

Address / Location (Shared Dimension)

Physical addresses for providers and facilities. The agreed direction is a shared address dimension (junk dimension) reused across both the provider and eligibility domains. Key complexities: sole-proprietor doctors have their own address; facilities have multiple locations; a doctor linked to a practice does not inherit all practice addresses — they may only work at two of ten locations. This requires bridge tables: T1-to-address and T2-to-address, with the linkage at the practice + specific address level.

Payer-Submitted Provider Data (Green/Orange Tables — Client Data)

This is the provider data that payers send in their files. It includes the payer's own proprietary client provider ID, their version of the provider's name, specialty, and address. This data must be stored verbatim and returned on authorization files — payers expect exact round-trip fidelity. The grain of the client provider ID varies per payer: some key by provider alone, others by provider + practice + physical location.

NPPES-to-Client Linkage

The T1 Link and T2 Link tables join the NPPES reference data to the payer-submitted data using NPI as the sole matching key. For V1, no fuzzy matching (name, specialty, zip code) is performed — it's a straight NPI join. Match score and match method fields are unnecessary and will be removed. This linkage is critical for enabling network checks and display enrichment, but the client's original data is what flows back on auths.

ClientT1 - ClientT2 Practice Affiliation

This represents which doctor (Type 1) works at which facility/practice (Type 2). This linkage comes exclusively from client data, not NPPES. The relationship is many-to-many and must be at the address level — a doctor affiliated with a practice may only work at specific locations within that practice. This is purely an employment/affiliation relationship, completely independent from network participation.

Network Participation

The highest-volume and most complex category. Networks represent contractual agreements between providers/facilities and payers. A single provider can belong to hundreds of networks (Cory estimated 80M+ records). Key clarification from the meeting: network participation is independent from T1-T2 affiliation. Both doctors and facilities independently link to networks. A shared network dimension table holds the network definitions, with separate bridge tables linking T1-to-network and T2-to-network. Networks drive contracting, claims payment, member benefit eligibility, and in/out-of-network determinations.

Site of Care

Represents payer-preferred vs. requested treatment locations. A payer may deny a request at one facility and redirect to a preferred lower-cost alternative. Currently implemented as a small Excel-based dictionary for one client. BA requirements are still in progress. This category needs a formal place in the model but its structure is deferred until requirements are finalized.

Risks

The provider domain carries risks concentrated in identity resolution, data volume, and unfinalized design decisions. The highest-severity items — NPI-absent handling, network bridge scalability, and unconfirmed versioning — are structural blockers to Silver schema finalization. A few key risks are listed in detail below.

NPI-absent provider records have no agreed handling strategy

Providers submitted without NPI cannot be linked to NPPES reference data, which breaks network checks, display enrichment, and downstream authorization workflows. The current Matis workaround — setting NPI to all-nines or requiring manual entry — is fragile and does not scale to multi-client onboarding. This affects every client that submits incomplete provider files.

Business impact: Cases referencing providers without NPI cannot complete network validation. Intake staff must manually intervene, creating bottlenecks and increasing error rates on authorization files returned to payers.

Working hypothesis: Provider records without NPI are loaded into the client provider table but flagged with a source status (e.g., NPI_MISSING). These records participate in case processing using client-provided attributes only. Network checks are skipped and the case is flagged for manual review. NPPES linkage is not attempted. If the client subsequently provides the NPI, the record is updated and linkage is retroactively established.

What this unblocks: ETL can load all client provider records without rejection. The application has a clear rule: NPI_MISSING = manual network verification. No data is silently discarded.

What could invalidate it: If the volume of NPI-absent records is high for certain clients, the manual review burden becomes unsustainable. Alternatively, if payers contractually require NPI on all auth responses, records without NPI may need to be rejected at ingestion rather than loaded with a flag.

NPPES data currency is unverified

The system design designates NPPES as the provider master data source, the data may not be consistently current. If NPPES records are stale, provider names, addresses, and specialties displayed in the application could be incorrect. No one on the team has recently validated the NPPES update cadence against actual data freshness.

Business impact: Clinical staff see outdated provider information during case processing. Incorrect addresses affect place-of-service validation. Stale taxonomy codes could mislabel provider specialties, undermining trust in the platform and creating audit findings.

Working hypothesis: NPPES weekly bulk files and incremental updates are ingested on a weekly cadence into Databricks Bronze. A freshness check compares a sample of recently updated NPI records against the CMS NPPES API to validate that the bulk file reflects current state. Freshness metrics are tracked as a DQ dimension on the provider reference dataset.

What this unblocks: The team can proceed with NPPES as the reference source with confidence. Freshness monitoring provides an early warning if data quality degrades.

What could invalidate it: If NPPES bulk files consistently lag real-world changes by weeks or months, the platform may need a supplementary real-time lookup against the CMS NPPES API for active case processing, adding infrastructure complexity.

Network participation data volume creates scalability risk

Each provider can link to hundreds of networks across clients. The network bridge could reach 80M+ records. The current proposed design with separate T1 and T2 tables doubles the bridge infrastructure — separate network-doctor and network-facility bridge tables, each with their own history. This is the largest data volume driver in the provider domain.

Business impact: Network checks are on the critical path for every prior authorization case. If the network bridge is slow to query, case processing latency increases. Storage and compute costs scale with record count. ETL refresh windows may not accommodate full reloads at this volume.

Working hypothesis: A single network participation bridge table links the shared network dimension to providers via provider_id (covering both T1 and T2 if consolidated). The bridge is partitioned by payer_id and Z-ordered on provider_id + network_id for query performance. Incremental loads use CDC rather than full replacement. Volume estimates are validated against actual client data before Silver schema is finalized.

What this unblocks: A single bridge table halves the maintenance surface. Partitioning and Z-ordering keep network checks within latency SLAs. Volume validation provides a factual basis for compute sizing.

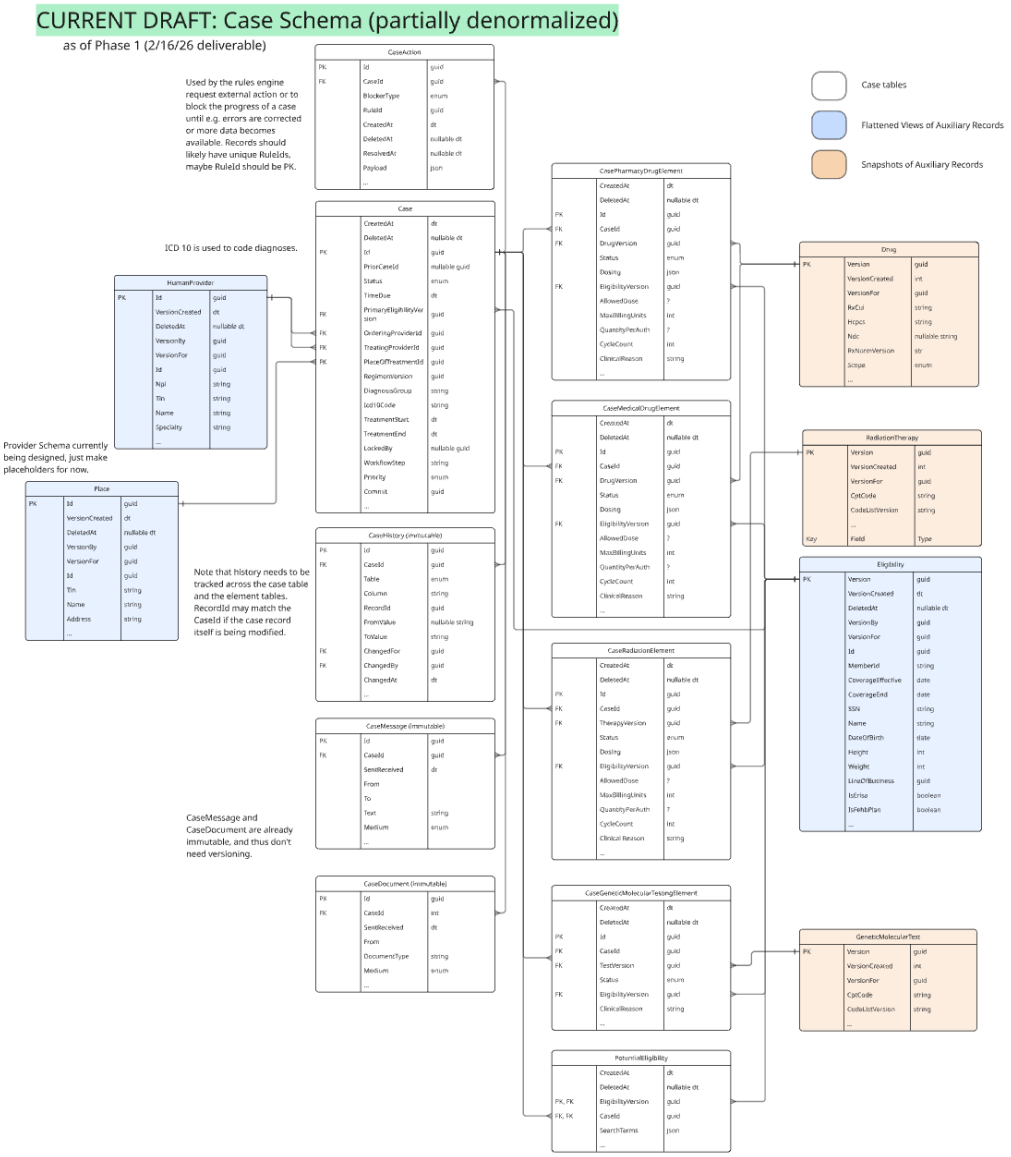
What could invalidate it: If T1 and T2 remain in separate tables, the bridge must be duplicated. If network participation data arrives as full-file replacements rather than deltas, incremental loading is not possible and full reloads at 80M+ rows may exceed SLA windows.

Cases domain

Data model ER diagram: Link to documentation

NewUM Case Engine Design: Link to documentation

The Case schema is the transactional core of NewUM. It stores the full lifecycle of a prior authorization case — clinical elements, workflow state, audit history, communications, and documents - in a partially denormalized, audit-safe structure. The schema is owned exclusively by the NewUM application layer; no analytic queries run against it directly.



Schema Layers

Layer	Color	Pattern
Core case tables	White	Mutable records; CaseHistory provides field-level audit
Auxiliary views	Blue	Flattened, soft-versioned lookups from Provider and Eligibility domains
Auxiliary snapshots	Orange	Versioned, append-only reference data from Drug/Radiation/GMT libraries

Core tables

Table	Purpose
Case	Root record. Holds live workflow state: status, priority, diagnosis, assigned providers, treatment group, deadlines, and consent. Mutable; audited via CaseHistory.
CaseHistory	Immutable field-level audit log across all mutable tables. Every column change appends a row recording table, column, old value, new value, timestamp, and the identity (user, ruleset, or service principal) that triggered the change. Full case state at any point time is reconstructed by replaying rows in ChangedAt order up to time.
CaseAction	Pending work queue for the rules engine. Records discrete blockers and actions — either rules-generated or externally triggered. Resolved via ResolvedAt rather than deleted; serves as both an operational control mechanism and an audit artifact.
CaseMessage	Append-only log of all inbound and outbound communications (fax, email, portal). Immutable; corrections append a new row.
CaseDocument	Append-only log of documents (lab results, auth letters, referrals). Same immutability contract as CaseMessage.
CaseMedicalDrugElement	Medical drugs attached to a case (administered in a facility). Mutable for status and dosing updates; references an immutable Drug snapshot.
CasePharmacyDrugElement	Pharmacy drugs dispensed directly to a patient. Handled separately from medical drugs due to distinct auth routing, RxNorm hierarchy depth, and dosing representation.
CaseRadiationElement	Radiation therapies on a case, identified by CPT code. Mutable for status and dosing; references an immutable RadiationTherapy snapshot.
CaseGeneticMolecularTestingElement	Genetic/molecular tests requiring PA, identified by CPT code. No dosing fields. References an immutable GeneticMolecularTest snapshot.
PotentialEligibility	Eligibility candidates surfaced during patient search, before a definitive eligibility record is assigned. Stores

	SearchTerms as JSON and links to EligibilityVersion and CaseId.

Auxiliary Records

Provider (HumanProvider, Place) and Eligibility records are flattened into the case schema to avoid spanning the full database during rules of engine fact-loading. Drug, RadiationTherapy, and GeneticMolecularTest records are stored as immutable snapshots at the time of assignment to the case — the source of truth for these is DrugsMS in OneUM.

Audit and Rollback

No data in the Case schema is ever hard deleted. The audit mechanisms by table:

Mechanism	Coverage
CaseHistory (old/new value per column)	Full field-level trail on Case and all mutable Case*Element tables
CreatedAt + DeletedAt on element tables	History of clinical elements added and removed
Snapshot versioning	Drugs, Radiation, GMT: snapshots stored with the case; Eligibility: snapshots in the Eligibility schema; Provider: change management TBD
Immutable CaseMessage / CaseDocument	Communications and documents cannot be retroactively altered
CaseAction with ResolvedAt	Full log of what blocked or triggered action on a case

Databricks Integration

The Cases domain is explicitly out of scope for the Data Team's medallion pipeline ownership, but operational case data is a registered ingestion source. NewUM pushes case records to Databricks Bronze on a daily batch cadence via the Data Team's REST ingestion API. The NewUM application never reads from Databricks at query time for case processing — the Case schema in PostgreSQL is the sole operational store. Downstream analytics consumers (case_summary, approval_funnel, turnaround_time, case_audit_trail) are served exclusively from Gold.

Databricks and NewUM Integration Mechanism

Bidirectional data exchange between Databricks and NewUM is required across multiple use cases, each with different latency and quality characteristics. No single mechanism serves all patterns. The working hypothesis is a layered approach:

#	Mechanism	Best Suited For	Key Advantages	Key Risks
A	Azure Event Hubs (Streaming/CDC)	High-volume continuous CDC from NewUM to Databricks	Near-real-time; decoupled; ordered per case; replay support	Requires EH infra; schema evolution coordination; consumer idempotency needed
B	Azure Service Bus (Message Queue)	Durable transactional delivery where ordering matters	Guaranteed delivery; dead-letter queues; per-message ack	Lower throughput; no native replay; less suited for CDC patterns
C	Databricks SQL Warehouse (JDBC/REST)	Background enrichment lookups (formulary, provider ref data)	No new infra; autoscaling; Unity Catalog RBAC	Read-only; seconds-range latency; runtime Databricks dependency
D	Databricks Lakebase (Serverless Postgres)	Low-latency transactional reads/writes of pre-computed results	Millisecond latency; standard Postgres protocol; native lakehouse integration	New service (2025); OAuth token rotation; cold-start risk after idle
H	ADLS Gen2 Direct File Access	Large infrequently changing reference datasets	Simple; no runtime dependency; well-supported .NET SDK	Batch-only; no Unity Catalog governance on direct reads
I	Azure Cache for Redis	High-frequency pre-computed lookups (eligibility, formulary)	Sub-ms reads; decouples compute from read timing	Cache invalidation complexity; Redis as new managed dependency

Data Exchange: NewUM → Databricks Bronze

Option Group 1: NewUM Exposes / Databricks Pulls

NewUM acts as data producer; Databricks manages ingestion timing, retry, and throughput. Bronze freshness is bounded by pull frequency.

Option	Description	Pros	Cons

CDC via Azure Event Hubs (current design)	NewUM DB publishes change events; Databricks Structured Streaming consumes	Near-real-time; ordered per case; durable replay; audit-friendly	Requires EH infrastructure; schema evolution coordination
REST API Polling	Databricks calls NewUM REST endpoints on a schedule	Decoupled from DB internals; respects API contracts	Rate limits; higher latency; must support delta queries
ADLS File Drop (Pull)	NewUM exports files to ADLS; Databricks picks up via Auto Loader	Resilient; easy schema inspection; good for reference data	Batch-only; deduplication required; file management overhead

Option Group 2: Databricks Exposes / NewUM Pushes

NewUM publishes state changes; Databricks provides receiving infrastructure. Lower latency, but NewUM must implement reliable outbound messaging.

Option	Description	Pros	Cons
Event Hubs (Push from App Layer)	NewUM app publishes domain events on state changes	Richer business context; real-time on write; decoupled transport	Requires instrumentation in NewUM; duplicate risk if app/DB desync
Databricks REST Ingest / Delta Sharing	NewUM pushes records to a Databricks ingestion endpoint	NewUM controls push timing; near-real-time	Tighter coupling to Databricks availability
Service Bus (Push from NewUM)	NewUM publishes to Service Bus topics	Built-in ordering, sessions, dead-lettering	Lower throughput ceiling; less native Spark integration

Working Hypothesis: batch ingestion via API exposed by Data Team

The Data Team exposes a REST ingestion API that NewUM calls to push records into Databricks Bronze. NewUM controls push timing—on commit, on schedule, or on domain event—while the Data Team owns validation, deduplication, and landing into Delta tables. This keeps NewUM's integration surface minimal (HTTP POST against a documented contract), centralizes ingestion logic on the

team that owns the lakehouse, and avoids NewUM taking a dependency on ADLS paths or Event Hubs infrastructure. The API contract serves as a versioned boundary between teams, simplifying schema evolution coordination. If throughput requirements outgrow synchronous HTTP, the API can front a Service Bus or Event Hubs backend without changing NewUM's calling pattern.

Medalion Layer Design

The NewUM Data team owns delivery of raw NewUM data into the Databricks Bronze layer. From there, the Onco Data team picks up the raw data and drives it through the medallion architecture — Bronze → Silver → Gold — applying the cleansing, conforming, and certification steps described below.

Once the Onco team publishes Gold-layer datasets purpose-built for NewUM, the NewUM team consumes those certified tables and passes the data into the NewUM application. The medallion layer descriptions below are included to paint the end-to-end picture; the transformation logic within Silver and Gold is owned and executed by the Onco Data team.

Bronze Layer

Raw ingestion tier storing full-fidelity, append-only copies of all source events with minimal transformation. Bronze is the system of record for replays, audits, and root-cause analysis. Each CDC event is stored as a row with metadata columns (ingestion timestamp, source batch ID, event schema version) appended by the ingestion framework.

Silver Layer

Transforms Bronze into a cleansed, deduplicated, conformed domain model. Applies type casting, SCD Type 2 versioning, MERGE-based deduplication, referential integrity validation, and PII tokenisation verification. Non-conforming rows route to quarantine (silver.quarantine.<entity_name>). Key Silver transformations include: dosing JSON decomposed into typed columns, CDC events deduplicated on PK + version key, and RI validated against Silver dimension tables before MERGE.

Gold Layer

Purpose-built datasets for specific consumer use cases. Gold tables are certified, pass all DQ gates, and are rebuilt on a scheduled cadence from Silver. Gold is the only layer directly accessible to BI analysts, payer consumers, and external reporting tools.

Unity Catalog Structure

Data assets are organized by domain: payer (PDL, formulary, NP rules), drugs (DrugsMS catalog), eligibility (payer feeds), providers (provider master data), and newum (operational case and drug data). Naming convention: <domain>_<environment> (e.g., newum_prod, providers_staging). Final naming to be confirmed during Unity Catalog governance setup.

Schema Evolution Approach

All schema changes should follow the Expand-Contract pattern. In Expand phase, new columns are added as nullable with no default, or with backward-compatible defaults. Existing queries continue to work. In Contract phase (after all consumers have migrated), NOT NULL constraints are added, and old columns are deprecated. Enum additions require a migration file, a rollback plan, and sign-off from the Schema Owner.

Metrics, Monitoring & Alerting

Data Quality

Six DQ dimensions are enforced: completeness (null rate thresholds), accuracy (domain/enum validation), consistency (referential integrity), timeliness (lag monitoring), uniqueness (duplicate detection), and validity (format/range checks). Tooling: Great Expectations for Silver entity suites, dbt tests for Gold relational tests, Delta CHECK constraints as a storage-layer guard, and DLT expectations where Bronze→Silver uses Delta Live Tables.

Quarantine & Alerting

Rows failing DQ at Silver are routed to quarantine tables (silver.quarantine.<entity_name>) with failure metadata. Alert thresholds: >1% quarantined → P3 Slack; >5% → P2 PagerDuty; >10% or PII-sensitive failure → P1 PagerDuty + incident creation. Quarantine rows retained 90 days; owning engineer triages P1/P2 within 24 hours.

Job & Pipeline Monitoring

Each Databricks job is tagged (team, pipeline, environment, owner) for cost attribution. Performance baselines (P50/P95 run duration, DBU consumption, rows processed) are established at launch; >20% deviation triggers an alert. A centralized observability dashboard tracks pipeline run duration trends, DQ pass/fail rates, quarantine trends, CDC lag, DBU cost by pipeline, and SLA breach events.

Deployment, Integration and Rollout

CI/CD Process

Trunk-based development with short-lived feature branches. The main branch is protected; all changes enter via PR. Environment promotion: main → staging (automatic on merge), staging → prod (manual approval gate). Databricks Asset Bundle (DAB) configuration stored per environment.

PR Review Gates

Minimum 1 Data Engineer approval for standard changes; 2 approvals for schema-breaking, Gold restructure, or security changes. Automated CI checks: pytest unit tests, Databricks integration tests, dbt tests, GX suite dry-run, linting (ruff/black), and schema diff review. Schema diff is posted as a PR comment.

Automated Testing

- Unit tests: pytest covering transformation logic, DQ checks, CDC event parsing. 80% line coverage gate.
- Integration tests: End-to-end Bronze→Silver→Gold on dedicated Databricks test workspace with synthetic data.
- dbt tests: not_null, unique, accepted_values, relationships, custom macros run in CI against test schema.
- GX dry-run: Suites run against last N days of production data via Delta Sharing to validate before merge.
- Schema evolution tests: Verify additive changes handled transparently by Auto Loader.
- Data contract tests: Validate inbound event schemas against documented JSON/Avro schemas before Bronze write.
- Idempotency tests: Confirm re-running any pipeline on same input produces identical output.

Deployment Pipeline

PR merge to main triggers ADO pipeline + Airflow CI workflow → unit/integration/dbt/GX/linting → on pass: DAB deploy to Staging + smoke tests → manual approval gate → production deployment via databricks bundle deploy --target prod → post-deploy smoke test confirms first successful execution.

Rollback Strategy

Delta Time Travel: Silver and Gold tables retain 30 days of history; rollback via RESTORE TABLE. Job definition rollback via re-deployment of prior Git-stored JSON snapshot. Hotfix branches bypass staging but require 2-engineer approval and same-day post-incident review.

Rollout

The production rollout follows an environment promotion path: development → staging → production. Staging deploys are automatic on merge; production requires Data Team Lead approval.

Post-deploy smoke tests must pass before the pipeline is considered live. The rollout will be coordinated with the OncoHealth Data team, who will consume the finalized data models and build Gold-layer assets.

Airflow Orchestration

Airflow is the canonical orchestration layer for all batch and hybrid pipelines. Design principles: one DAG per logical pipeline; composable DAGs linked via TriggerDagRunOperator or Datasets API; owner tag + Slack channel on every DAG; environment-specific values injected via Airflow Variables/Connections; UTC cron schedules; max_active_runs=1 for production ingestion; catchup=False by default.

Databricks jobs are invoked via DatabricksRunNowOperator with PAT or OAuth M2M credentials stored in Airflow Connections. Job cluster configuration is defined in Databricks job definitions, not Airflow. Run IDs are persisted in XCom for audit.

Pipeline Tier	Retries	Retry Delay	On Exhaustion
Critical (CDC, Billing, Clinical Ops)	3	5 min (exponential)	P1 PagerDuty + incident auto-created + SLA breach notification
Standard (Gold refresh, Payer exports)	2	10 min	P2 PagerDuty + Slack alert #data-eng-alerts
Low-priority (ML features, DQ trending)	1	30 min	P3 Slack alert only

Airflow metrics are exported to Prometheus → Grafana. Dead-letter monitoring alerts on tasks stuck >15 minutes. DAG run history retained 90 days; logs exported to Azure Monitor for longer retention

Scalability

- Design ceiling: 500,000 cases/day (10x current peak of 50,000) without architectural change.
- Databricks clusters use autoscaling with defined min/max worker range per pipeline tier.
- Delta table compaction (OPTIMIZE + ZORDER) runs weekly for Silver/Gold tables exceeding 100M rows; target 256 MB per file.
- Event Hub partitions sized for peak throughput with 20% headroom.

Testing

The test strategy spans six layers: unit tests (pytest, 80% coverage), integration tests (end-to-end on dedicated Databricks workspace), dbt relational tests, Great Expectations suite validation, schema evolution tests (Auto Loader compatibility), and data contract tests (inbound schema

validation). Idempotency tests verify replay safety. All test types are integrated into the CI/CD pipeline gate; no pipeline reaches production without passing the full suite.

Data Migration

MATIS to NewUM Migration

Only active and open cases are migrated at cutover. Closed and historical cases remain in MATIS for direct reference. Every migrated record carries Legacy Case ID (original MATIS identifier) and Legacy Source System (set to "MATIS"; designed to accommodate future sources like SmartUM).

Re-opened cases (appeals, renewals) after cutover result in a shell case record in NewUM linked to the original Legacy Case ID. Post-cutover, NewUM is the sole system of record for all case activity.

Legacy identifiers are leveraged by reporting ETL and data warehouse processes to link NewUM records back to MATIS history, enabling continuity in longitudinal reporting across the transition.

Out of scope for migration: closed/finalized MATIS cases, historical documents not attached to active cases, and SmartUM records (future phase).

Data Management

Data Retention Policy

Layer / Dataset	Retention	Deletion Mechanism	Compliance Driver
Bronze (all tables)	7 years	ADLS Gen2 lifecycle + Delta VACUUM	HIPAA minimum for PHI-adjacent data
Silver (operational)	7 years	Delta VACUUM (retain 7yr history)	HIPAA; payer contract obligations
Gold (reporting)	3 years	Partition drop + VACUUM	Business analytics policy
Compliance Audit Trail	10 years	ADLS Gen2 immutable (WORM) + append-only Delta	HIPAA; regulatory audit requirement
Quarantine tables	90 days	Automated partition drop job	Operational (no long-term value)

Right-to-erasure (RTBF): Where required by payer contract or regulation, a defined workflow handles tokenisation record deletion plus Delta REPLACE WHERE on the affected member token. Process SLA: 30 days from request.

Schema Versioning & Evolution

All Bronze schema changes are captured in the Avro Schema Registry. Compatible changes (new nullable columns) are auto-applied via Auto Loader with mergeSchema=true. Breaking changes (renames, type changes, drops) require advance notice, a Bronze migration script, Silver/Gold migration jobs, and consumer notification before deployment. Silver and Gold schema changes follow CI/CD with mandatory schema diff review. Delta table schema evolution is disabled by default; only explicitly approved changes are applied. A schema changelog is maintained in /docs/schema-history/ in the data platform repo.

Data Contracts

Every ingestion source must have a documented, owner-confirmed contract before production onboarding. Contract formats: NewUM CDC uses JSON event envelope schema versioned in Git (with optional Avro); DrugsMS uses OpenAPI spec with version field per response record; Payer Eligibility uses payer-specific file specs with JSON schema in /docs/contracts/. Breaking-change protocol: 7-day advance notice, coordinated cutover, and consumer notification.

Security and Compliance Concerns

The platform operates in a HIPAA-regulated healthcare prior-authorization context. PHI is tokenised at the Bronze write layer before any downstream processing. The tokenisation service must support the <5-minute CDC latency target. Unity Catalog enforces fine-grained RBAC including column- and row-level security. ADLS Gen2 storage firewall policies, Azure Key Vault for secret management, and Azure Active Directory for identity federation are core security infrastructure.

HIPAA BAA coverage is a mandatory filter for all third-party tooling evaluations (e.g., Fivetran, Airbyte). Oncology data must not pass through third-party infrastructure without explicit BAA verification.

Audit requirements: the gold.compliance.case_audit_trail table is stored in ADLS Gen2 immutable storage (WORM) with append-only Delta, retained for 10 years. All Airflow DAG runs, Databricks job executions, and data access events are logged and exported to Azure Monitor Log Analytics.

PHI field classification (beyond SSN, DOB, name) requires a data-classification review with Legal/Compliance before the Silver schema is finalised. This is tracked as Open Question #9.

Scope / Constraints

Scope

- All ingestion pipelines from NewUM and related systems (Eligibility, Providers, Members, Drugs, Regimens).
- Outbound feeds and API-served datasets from platform processing.

- Medallion (Bronze/Silver/Gold) data model for NewUM domain entities.
- Databricks compute, Airflow orchestration, DQ, CI/CD, and monitoring standards.
- Functional and non-functional requirements: security, scalability, retention, and DR.

Out of Scope

- Cases domain (ownership TBD separately).
- SmartUM records (future phase).
- Closed/finalized MATIS cases.
- Admin Portal configuration management (consumed, not maintained by data platform).

Reliability & Availability

Component	Target	Resilience Mechanism
CDC Ingestion (EH → Bronze)	99.9%/month	Consumer group rebalancing; DLQ for unprocessable events; auto-restart
Silver Processing Jobs	99.5%/month	Airflow retry logic; Delta ACID prevents partial writes
Gold Refresh Jobs	99.5%/month	Delta Time Travel revert; consumers read prior certified version until refresh
Databricks SQL (BI endpoint)	99.9%/month	Serverless SQL auto-scales; zero cluster cold-start for BI
Airflow Scheduler	99.9%/month	AKS active-standby; failover < 2 min; DAG code in Git

Disaster Recovery

Component	RPO	RTO	Recovery Mechanism
Bronze Delta Tables (ADLS)	< 1 hour	< 4 hours	Azure Storage GRS/RA-GRS; Delta re-register in DR workspace

Silver / Gold Delta Tables	< 4 hours	< 8 hours	ADLS GRS + pipeline re-run from Bronze; Time Travel for clean checkpoint
Airflow (on AKS)	< 1 hour	< 2 hours	AKS cluster in DR region; Helm chart; DAG code in Git; Postgres geo-redundant backup
Databricks Workspace Config	< 1 day	< 4 hours	Databricks Asset Bundles in Git; redeploy to DR workspace
Unity Catalog Metadata	< 4 hours	< 8 hours	Databricks-managed storage; DR workspace linked to ADLS GRS secondary

DR runbook tested quarterly via tabletop exercise; live failover test to DR Databricks workspace performed annually.

Constraints

- No analytic queries against operational databases; all analytics from Databricks Gold/Silver.
- Direct SQL/JDBC to NewUM operational DB is excluded as an anti-pattern.
- Airflow is the sole orchestration layer; ADF introduces a redundant orchestration surface.
- HIPAA/BAA compliance is a first-class filter for all third-party tooling.
- Event Hub schema changes must be backward-compatible for zero-downtime ingestion.

Glossary

Open Questions

- Integration Mechanism: Which bidirectional integration pattern(s) from the evaluation are confirmed? Gates Silver write-back and NewUM read-back design.
- Matis API Coverage: Does existing API return all required fields (EadlFlag, GPI, BenefitType, RxCUI variants, RepresentativeNdc)?
- Consumer Registry: Incomplete. Discovery sessions required with Analytics, Finance, Clinical Ops, Compliance, and NewUM Eng.
- Tooling: Standardise on DatabricksRunNowOperator with DABs as single source of truth for job definitions?
- Cost: Approved DBU budget per environment and per pipeline tier?
- DR: Azure DR region selection and standby model (warm vs cold)?
- Schema Versioning: Formal format (e.g., v<major>.<minor>) and version bumping rules?
- Compliance: Which PHI fields beyond SSN, DOB, name are in scope for HIPAA tokenisation?
- Data Model: 30 HIGH-priority schema gaps from Feb 2026 Gap Analysis — which are resolved before V1.0?

- Monitoring: Azure Monitor / App Insights as enterprise observability platform, or Databricks SQL dashboards as primary surface?

Assumptions

#	Assumption	Impact if Invalid
A1	Databricks is the agreed compute platform. No competing platforms (Snowflake, Redshift) are in scope.	Major rework of data modeling and processing sections.
A2	Unity Catalog is available and approved as the governance layer.	RBAC and data lineage approach must be redesigned.
A3	Apache Airflow on AKS is the agreed orchestration layer.	Orchestration section must be rewritten for Databricks Workflows.
A4	NewUM operational DB CDC via Azure Event Hubs is approved and feasible.	Bronze ingestion reverts to scheduled batch SQL pull with higher latency.
A5	Azure is the cloud provider (Databricks, ADLS Gen2, Event Hubs, Key Vault, AKS).	All Azure-specific infrastructure references require revision.
A6	Schema gaps from Feb 2026 Gap Analysis resolved before Silver layer finalization.	Silver models will contain workarounds requiring backfill migration.
A7	PII tokenisation service is available at Bronze ingestion time with < 5-min SLA.	PII handling strategy at Bronze must be redesigned.
A8	Matis API covers all NewUM-required drug catalog fields.	Silver drug dimension must handle missing fields via derived columns.
A9	Great Expectations (OSS) is acceptable as the DQ framework.	DQ tooling section must align with enterprise platform selection.